

DOCUMENTO DE ARQUITETURA

Sistema de Gerenciamento de Estoque

Engenharia de Software

Marcelo Ramalho Carnaúba
Lucas Gabriel Brasileiro da Silva
Alberto Costa Santos Sobrinho
Marco Valerio Wichmann Da Silva Junior
Matheus Albuquerque Santa Ritta

Versão 1.0 — 2026

1. Introdução

Este documento descreve a arquitetura de software do Sistema de Gerenciamento de Estoque, projeto acadêmico do 5º período. O objetivo é apresentar as decisões tecnológicas, estrutura em camadas, componentes principais e modelagem de dados adotados pela equipe.

Tecnologias principais: Python 3.11 + FastAPI (Backend) · React.js + TypeScript (Frontend) · PostgreSQL (Banco de Dados)

Estilo arquitetural: REST API com separação em camadas (Frontend / Backend / Banco de Dados)

Versão do documento: 1.0 — Entrega acadêmica 2026

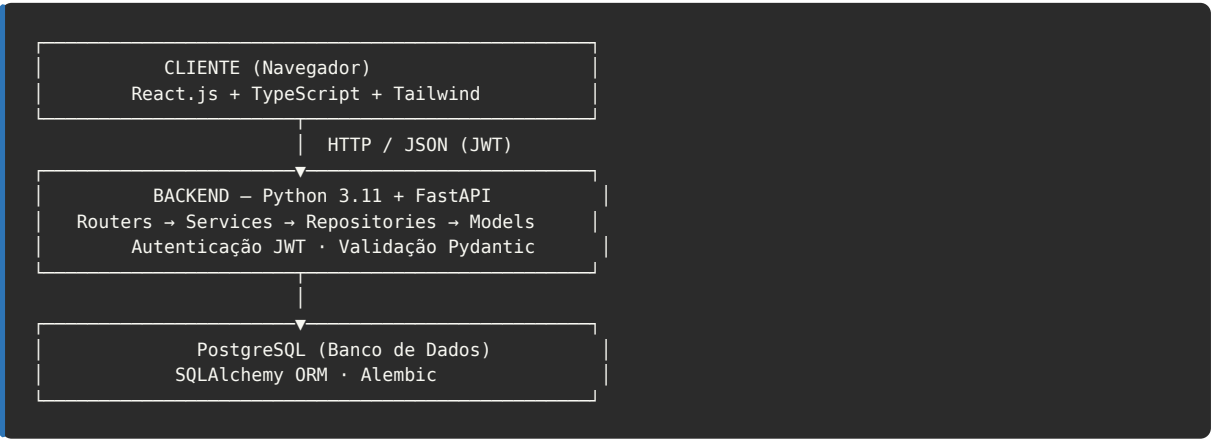
1.1 Controle de Versões

Versão	Data	Autor	Descrição
1.0	2026	Equipe de Desenvolvimento	Elaboração inicial do documento de arquitetura

2. Visão Geral da Arquitetura

O sistema é uma aplicação web com três camadas: Frontend SPA (React.js), Backend API RESTful (FastAPI) e banco de dados relacional (PostgreSQL). A comunicação entre cliente e servidor ocorre via HTTP/JSON com autenticação por JWT. O ambiente é containerizado com Docker Compose.

2.1 Diagrama de Camadas



3. Stack Tecnológica

Camada	Tecnologia	Função
Frontend	React.js + TypeScript	Interface do usuário (SPA)
Estilização	Tailwind CSS + shadcn/ui	Design system e componentes de UI
Estado / Cache	React Query + Zustand	Cache de dados do servidor e estado global
Roteamento	React Router v6	Navegação client-side
Backend	Python 3.11 + FastAPI	API RESTful assíncrona
ORM / Migrations	SQLAlchemy 2.0 + Alembic	Mapeamento objeto-relacional e versionamento do schema
Validação	Pydantic v2	Schemas de entrada/saída e validação de dados
Autenticação	JWT + bcrypt	Autenticação stateless e hash de senhas
Banco de Dados	PostgreSQL	Persistência relacional
Containerização	Docker + Docker Compose	Ambiente de desenvolvimento e deploy
Versionamento	Git + GitHub	Controle de versão

4. Estrutura do Backend – FastAPI

O backend segue arquitetura em camadas com separação de responsabilidades: routers recebem requisições, services contêm regras de negócio e repositories executam queries no banco. Cada módulo de negócio agrupa seus próprios arquivos.

4.1 Organização de Pastas

```
backend/
├── app/
│   ├── main.py           # Inicialização do FastAPI
│   ├── core/
│   │   ├── config.py     # Configurações e variáveis de ambiente
│   │   ├── security.py   # JWT e bcrypt
│   │   └── database.py   # Conexão SQLAlchemy
│   └── modules/
│       ├── produtos/
│       │   ├── router.py  # Endpoints REST
│       │   ├── service.py # Regras de negócio
│       │   ├── repository.py# Queries SQLAlchemy
│       │   ├── models.py  # Tabelas ORM
│       │   └── schemas.py # Pydantic (Request/Response)
│       ├── estoque/
│       ├── movimentacoes/
│       ├── inventarios/
│       └── usuarios/
├── migrations/           # Alembic
├── tests/
├── Dockerfile
└── requirements.txt
```

4.2 Fluxo de Requisição

Toda requisição percorre: **Router** (validação Pydantic + auth JWT) → **Service** (regras de negócio) → **Repository** (query SQLAlchemy) → resposta JSON ao cliente.

5. Estrutura do Frontend — React.js

```
frontend/
├── src/
│   ├── main.tsx
│   ├── App.tsx           # Roteamento (React Router v6)
│   ├── components/      # Componentes reutilizáveis
│   │   ├── ui/          # shadcn/ui
│   │   └── layout/      # Sidebar, Topbar
│   ├── pages/           # Uma pasta por módulo
│   │   ├── Dashboard/
│   │   ├── Produtos/
│   │   ├── Estoque/
│   │   ├── Movimentacoes/
│   │   ├── Inventario/
│   │   └── Alertas/
│   ├── services/        # Chamadas Axios à API
│   ├── hooks/           # Custom Hooks (React Query)
│   ├── store/           # Zustand (sessão do usuário)
│   └── types/           # Interfaces TypeScript
├── vite.config.ts
└── package.json
```

Autenticação: Access Token JWT armazenado em memória (Zustand), Refresh Token em cookie HttpOnly. Interceptor Axios renova o token automaticamente no HTTP 401. Rotas protegidas por componente ProtectedRoute.

6. Modelagem de Dados — PostgreSQL

Banco relacional normalizado com integridade referencial e suporte a transações ACID. Schema versionado via Alembic.

Tabela	Descrição
produtos	Cadastro de produtos — SKU, nome, unidade, configurações de lote/validade/série
depositos	Depósitos e centros de distribuição
localizacoes	Endereçamento físico dentro de um depósito
usuarios	Credenciais e perfis de acesso (Admin, Estoquista, Gerente)
saldos_estoque	Posição consolidada por produto / depósito / localização
movimentacoes	Registro imutável de entradas, saídas, transferências e ajustes
lotes	Lotes com data de validade e rastreabilidade
inventarios	Inventários físicos e divergências apuradas
ajustes	Ajustes aprovados com justificativa formal
alertas	Alertas de estoque mínimo e vencimento

Decisões relevantes: soft delete em produtos/fornecedores (campo ativo); tabela movimentacoes append-only como fonte de verdade do saldo; SELECT FOR UPDATE para evitar race conditions em operações concorrentes.

7. Segurança

Controle	Implementação
Autenticação	JWT — Access Token (15 min) + Refresh Token (7 dias)
Autorização	RBAC — perfis Admin, Estoquista e Gerente com permissões por rota
Senhas	Hash bcrypt — nunca armazenadas em texto plano
Validação	Pydantic v2 em todos os endpoints — rejeição de payloads inválidos
CORS	Origens permitidas configuradas explicitamente
Variáveis sensíveis	Gerenciadas via .env — nunca no repositório